ZOMATO ANALYTICS

BY GROUP-6

Presented By:

- Vishal Waikar
- Reshma Gorde
- Destiny B.B
- Vansh Singhal
- Cheethirala Varshini
- Anshuman Patle
- Divyanka Khas

Project Overview

This project transforms raw Zomato data into actionable insights to support informed business decisions.

- Analyze Zomato restaurant dataset
- Build dashboards in Excel, Power BI, and Tableau
- Identify trends in cities, cuisines, ratings, and pricing
- Perform backend analysis using MySQL

END-TO-END PROJECT WORKFLOW

This structured workflow ensured data accuracy and enabled reliable, actionable business insights.

- Data Understanding
- Data Cleaning & Transformation (Excel Power Query)
- KPI Identification
- Visualization & Dashboarding
- Business Insights & Recommendations
- MySQL Case Studies (Used MySQL for structured data analysis)

SQL CASE STUDY 1

#1 CREATE EMP TABLE

```sql
select * from emp;
```

empno	ename	job	mgr	hiredate
7369	SMITH	CLERK	7902	1980-12-17
7499	ALLEN	SALESMAN	7698	1981-02-20
7521	WARD	SALESMAN	7698	1981-02-22
7566	JONES	MANAGER	7839	1981-04-02
7654	MARTIN	SALESMAN	7698	1981-09-28
7698	BLAKE	MANAGER	7839	1981-05-01
7782	CLARK	MANAGER	7839	1981-06-09
7788	SCOTT	ANALYST	7566	1987-04-19
7839	KING	PRESIDENT	NULL	1981-11-17
7844	TURNER	SALESMAN	7698	1981-09-08
7876	ADAMS	CLERK	7788	1987-05-2

#2 CREATE DEPT TABLE

```sql
select * from dept;
```

deptno	dname	loc
10	OPERATIONS	BOSTON
20	RESEARCH	DALLAS
30	SALES	CHICAGO
40	ACCOUNTING	NEW YORI
NULL	NULL	NULL

#3.LIST THE NAMES AND SALARY OF THE EMPLOYEE WHOSE SALARY IS GREATER THAN 1000

```sql
SELECT ename, sal
FROM emp
WHERE sal > 1000;
```

ename	sal
ALLEN	1600.00
WARD	1250.00
JONES	2975.00
MARTIN	1250.00
BLAKE	2850.00
CLARK	2450.00
	3000.00
	5000.00

#4. LIST THE DETAILS OF THE EMPLOYEES WHO HAVE JOINED BEFORE END OF SEPT 81.

```sql
SELECT *
FROM emp
WHERE hiredate < '1981-10-01';
```

empno	ename	job	mgr	hiredate	sal
7369	SMITH	CLERK	7902	1980-12-17	800.00
7499	ALLEN	SALESMAN	7698	1981-02-20	1600.00
7521	WARD	SALESMAN	7698	1981-02-22	1250.00
7566	JONES	MANAGER	7839	1981-04-02	2975.00
7654	MARTIN	SALESMAN	7698	1981-09-28	1250.00
7698	BLAKE	MANAGER	7839	1981-05-01	2850.00
7782	CLARK	MANAGER	7839	1981-06-09	2450.00
7844	TURNER	SALESMAN	7698	1981-09-08	1500.00

#5.LIST EMP NAMES HAVING I AS SECOND CHARACTER

```sql
SELECT ename
FROM emp
WHERE ename LIKE '_I%';
```

ename
KING
MILLER

#6.LIST EMP NAME, SALARY, ALLOWANCES (40% OF SAL), P.F. (10 % AND NET SALARY. ALSO ASSIGN THE ALIAS NAME FOR THE COLU

```sql
    ename AS Employee_Name,
    sal   AS Salary,
    sal * 0.40 AS Allowances,
    sal * 0.10 AS PF,
    sal + (sal * 0.40) - (sal * 0.10) AS Net_Salary
FROM emp;
```

Employee_Name	Salary	Allowances	PF	Net_Salary
SMITH	800.00	320.0000	80.0000	1040.0000
ALLEN	1600.00	640.0000	160.0000	2080.0000
WARD	1250.00	500.0000	125.0000	1625.0000
JONES	2975.00	1190.0000	297.5000	3867.5000
MARTIN	1250.00	500.0000	125.0000	1625.0000
BLAKE	2850.00	1140.0000	285.0000	3705.0000
CLARK	2450.00	980.0000	245.0000	3185.0000
SCOTT	3000.00	1200.0000	300.0000	3900.0000
KING	5000.00	2000.0000	500.0000	6500.0000
TURNER	1500.00	600.0000	150.0000	1950.0000

#7. LIST EMP NAMES WITH DESIGNATIONS WHO DOES NOT REPORT TO ANYBODY

```sql
SELECT ename, job
FROM emp
WHERE mgr IS NULL;
```

ename	job
KING	PRESIDENT

#8.LIST EMPNO, ENAME AND SALARY IN THE ASCENDING ORDER OF SALARY.

```sql
FROM emp
ORDER BY sal ASC;
```

Result Grid | Filter Rows:

empno	ename	sal
7369	SMITH	800.00
7900	JAMES	950.00
7876	ADAMS	1100.00
7521	WARD	1250.00
7654	MARTIN	1250.00
7934	MILLER	1300.00
7844	TURNER	1500.00
7499	ALLEN	1600.00
7782	CLARK	2450.00
7698	BLAKE	2850.00

#9.HOW MANY JOBS ARE AVAILABLE IN THE ORGANIZATION ?

```sql
SELECT COUNT(DISTINCT job) AS Total
FROM emp;
```

Result Grid | Filter Rows: | Exp

Total_Jobs
5

#.10 DETERMINE TOTAL PAYABLE SALARY OF SALESMAN CATEGORY

```sql
SELECT SUM(sal) AS Total_Payabl
FROM emp
WHERE job = 'SALESMAN';
```

Result Grid | Filter Rows:

#11.LIST AVERAGE MONTHLY SALARY FOR EACH JOB WITHIN EACH DEPARTMENT

```sql
SELECT
    deptno,
    job,
    AVG(sal) AS Avg_Monthly_Salary
FROM emp
GROUP BY deptno, job;
```

Result Grid | Filter Rows: | Export:

deptno	job	Avg_Monthly_Salary
20	CLERK	950.000000
30	SALESMAN	1400.000000
20	MANAGER	2975.000000
30	MANAGER	2850.000000
10	MANAGER	2450.000000
20	ANALYST	3000.000000
10	PRESIDENT	5000.000000
30	CLERK	950.000000
10	CLERK	1300.000000

#12.USE THE SAME EMP AND DEPT TABLE USED IN THE CASE STUDY TO DISPLAY EMPNAME, SALARY AND DEPTNAME IN WHICH THE EMPLOYEE IS WORKING

```sql
SELECT
    e.ename AS Employee_Name,
    e.sal   AS Salary,
    d.dname AS Department_Name
FROM emp e
JOIN dept d
ON e.deptno = d.deptno;
```

Result Grid | Filter Rows:

Employee_Name	Salary	Department_Nar
CLARK	2450.00	OPERATIONS
KING	5000.00	OPERATIONS
MILLER	1300.00	OPERATIONS
SMITH	800.00	RESEARCH
JONES	2975.00	RESEARCH
SCOTT	3000.00	RESEARCH
ADAMS	1100.00	RESEARCH
FORD	3000.00	RESEARCH
ALLEN	1600.00	SALES
WARD	1250.00	SALES
MARTIN	1250.00	SALES
BLAKE	2850.00	SALES
TURNER	1500.00	SALES
JAMES	950.00	SALES

#13.CREATE JOB GRADE TABLE

```sql
select * from job_grades;
```

Result Grid | Filter Rows:

grade	lowest_sal	highest_sal
A	0	999
B	1000	1999
C	2000	2999
D	3000	3999
E	4000	5000
NULL	NULL	NULL

#14.DISPLAY THE LAST NAME, SALARY AND CORRESPONDING GRADE.

```sql
SELECT e.ename AS Employe
FROM emp e
JOIN job_grades j
ON e.sal BETWEEN j.lowest
```

Result Grid | Filter Rows:

Employee_Name	Salary	Grade
SMITH	800.00	A
ALLEN	1600.00	B
WARD	1250.00	B
JONES	2975.00	C
MARTIN	1250.00	B
BLAKE	2850.00	C
CLARK	2450.00	C
SCOTT	3000.00	D

#15.DISPLAY THE EMP NAME AND THE MANAGER NAME UNDER WHOM THE EMP WORKS IN THE EMP REPORT TO MGR FORMAT

```
SELECT E.ENAME AS "Emp",M.ENAME AS "Report to Mgr"
FROM EMP E
LEFT JOIN EMP M ON E.MGR = M.EMPNO;
```

Result Grid | Filter Rows: | Export: | Wr

Emp	Report to Mgr
SMITH	FORD
ALLEN	BLAKE
WARD	BLAKE
JONES	KING
MARTIN	BLAKE
BLAKE	KING
CLARK	KING

#16.DISPLAY EMPNAME AND TOTAL SAL WHERE TOTAL SAL (SAL + COMM)

```
SELECT
    ENAME AS `Emp Name`,SAL + IFNULL(COMM, 0) AS `Total Sal`FROM EMP
LIMIT 1000;
```

Result Grid | Filter Rows: | Export:

Emp Name	Total Sal
SMITH	800.00
ALLEN	1900.00
WARD	1750.00
JONES	2975.00
MARTIN	2650.00
BLAKE	2850.00
CLARK	2450.00
SCOTT	3000.00
KING	5000.00
TURNER	1500.00

#.17 DISPLAY EMPNAME AND SAL WH EMPNO IS A ODD NUMBER

```
SELECT
    ENAME AS `Emp Name`,SAL AS `Salary`
FROM EMP
WHERE MOD(EMPNO, 2) = 1;
```

Result Grid | Filter Rows: | Ex

Emp Name	Salary
SMITH	800.00
ALLEN	1600.00

#18. DISPLAY EMPNAME , RANK OF SAL IN ORGANISATION , RANK OF SAL IN THEIR DEPARTMENT

```
SELECT
    ENAME AS `Emp Name`,
    RANK() OVER (ORDER BY SAL DESC) AS `Org Rank`,
    RANK() OVER (PARTITION BY DEPTNO ORDER BY SAL DESC) AS `Dept Rank`
FROM EMP;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Emp Name	Org Rank	Dept Rank
KING	1	1
CLARK	6	2
MILLER	9	3
SCOTT	2	1
FORD	2	1
JONES	4	3
ADAMS	12	4
SMITH	14	5

#19.DISPLAY TOP 3 EMPNAMES BASED ON THEIR SALARY

```
SELECT ENAME AS `Emp Name`,SAL AS `Salary`FROM EMP
ORDER BY SAL DESC LIMIT 3;
```

esult Grid | Filter

Emp Name	Salary
KING	5000.00
FORD	3000.00
SCOTT	3000.00

DISPLAY EMPNAME WHO HAS HIGHEST SALARY IN EACH DEPT

```
SELECT ENAME AS `Emp Name`,DEPTNO AS `Department`,SAL AS `Salary`
FROM (SELECT ENAME,DEPTNO,SAL,RANK() OVER (PARTITION BY DEPTNO OR
FROM EMP) AS ranked_emp WHERE sal_rank = 1;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Emp Name	Department	Salary
KING	10	5000.00
SCOTT	20	3000.00
FORD	20	3000.00
BLAKE	30	2850.00

SQL CASE STUDY 2

#1.CREATE TABLE SALESPEOPLE

```sql
select * from Salespeople;
```

snum	sname	city	comm
1001	Peel	London	0.12
1002	Serres	San Jose	0.13
1003	Axelrod	New york	0.10
1004	Motika	London	0.11
1007	Rafkin	Barcelona	0.15

#2. CREATE TABLE CUST

```sql
select * from Cust;
```

cnum	cname	city	rating	snum
2001	Hoffman	London	100	1001
2002	Giovanne	Rome	200	1003
2003	Liu	San Jose	300	1002
2004	Grass	Berlin	100	1002
2006	Clemens	London	300	1007

#3.CREATE TABLE ORDERS

```sql
select * from Orders;
```

onum	amt	odate	cnum	snum
3001	18.69	1994-10-03	2008	1007
3002	1900.10	1994-10-03	2007	1004
3003	767.19	1994-10-03	2001	1001
3005	5160.45	1994-10-03	2003	1002
3006	1098.16	1994-10-04	2008	1007
3007	75.75	1994-10-05	2004	1002
3008	4723.00	1994-10-05	2006	1001

#4.WRITE A QUERY TO MATCH THE SALESPEOPLE TO THE CUSTOMERS ACCORDING TO THE CITY THEY ARE LIVING.

```sql
SELECT s.sname AS Salesperson, c.cname AS Customer, s.city AS City
FROM Salespeople s JOIN Cust c ON s.city = c.city;
```

Salesperson	Customer	City
Motika	Hoffman	London
Peel	Hoffman	London
Serres	Liu	San Jose
Motika	Clemens	London
Peel	Clemens	London
Motika	James	London

#5.WRITE A QUERY TO SELECT THE NAMES OF CUSTOMERS AND THE SALESPERSONS WHO ARE PROVIDING SERVICE TO THEM

```sql
SELECT
    c.cname AS Customer_Name,
    s.sname AS Salesperson_Name
FROM Cust c
JOIN Salespeople s ON c.snum = s.snum;
```

Customer_Name	Salesperson_Name
Hoffman	Peel
Liu	Serres

#6.WRITE A QUERY TO FIND OUT ALL ORDERS BY CUSTOMERS NOT LOCATED IN THE SAME CITIES AS THAT OF THEIR SALESPEOPLE

```sql
SELECT
    o.onum AS Order_No, c.cname AS Customer,
    c.city AS Cust_City, s.sname AS Salesperson,
    s.city AS Sales_City FROM Orders o
JOIN Cust c ON o.cnum = c.cnum
JOIN Salespeople s ON o.snum = s.snum
```

Result Grid | Filter Rows: | Export: | Wrap Cell

Order_No	Customer	Cust_City	Salesperson	Sales_City
3007	Grass	Berlin	Serres	San Jose
3010	Grass	Berlin	Serres	San Jose
3009	Giovanne	Rome	Axelrod	New york
3002	Pereira	Rome	Motika	London
3001	James	London	Rafkin	Barcelona

#7.WRITE A QUERY THAT LISTS EACH ORDER NUMBER FOLLOWED BY NAME OF CUSTOMER WHO MADE THAT ORDER

```sql
SELECT
    o.onum AS Order_Number,
    c.cname AS Customer_Name
FROM Orders o
JOIN Cust c ON o.cnum = c.cnum;
```

Result Grid | Filter Rows: | Exp

Order_Number	Customer_Name
3003	Hoffman
3009	Giovanne
3005	Liu
3007	Grass
3010	Grass
3008	Clemens
3011	Clemens
3002	Pereira

#8.WRITE A QUERY THAT FINDS ALL PAIRS OF CUSTOMERS HAVING THE SAME RATING

```sql
SELECT
    a.cname AS Customer_1,
    b.cname AS Customer_2,
    a.rating AS Shared_Rating FROM Cust
JOIN Cust b ON a.rating = b.rating
WHERE a.cnum < b.cnum;
```

ult Grid | Filter Rows: | Export:

Customer_1	Customer_2	Shared_Rating
Hoffman	Grass	100
Hoffman	Pereira	100
Giovanne	James	200
Liu	Clemens	300
Grass	Pereira	100

```sql
    a.cname AS Customer_1, b.cname AS Customer_2,
    s.sname AS Shared_Salesperson
    FROM Cust a JOIN Cust b ON a.snum = b.snum
    JOIN Salespeople s ON a.snum = s.snum
    WHERE a.cnum < b.cnum;
```

sult Grid | Filter Rows: | Export: | Wra

Customer_1	Customer_2	Shared_Salesperson
Liu	Grass	Serres
Clemens	James	Rafkin

#9.WRITE A QUERY TO FIND OUT ALL PAIRS OF CUSTOMERS SERVED BY A SINGLE SALESPERSON

#10.WRITE A QUERY THAT PRODUCES ALL PAIRS OF SALESPEOPLE WHO ARE LIVING IN SAME CITY

```sql
    a.sname AS Salesperson_1,
    b.sname AS Salesperson_2, a.city AS City FROM Salespeople a
JOIN Salespeople b ON a.city = b.city
WHERE a.snum < b.snum;
```

lt Grid | Filter Rows: | Export: | Wrap Cell Content:

Salesperson_1	Salesperson_2	City
eel	Motika	London

#11. WRITE A QUERY TO FIND ALL ORDERS CREDITED TO THE SAME SALESPERSON WHO SERVICES CUSTOMER 2008

```sql
SELECT onum, amt, odate, cnum, snum
FROM Orders
WHERE snum = (SELECT snum FROM Cust WHERE cnum = 2008);
```

Result Grid | Filter Rows: | Edit: | Export/Imp

onum	amt	odate	cnum	snum
3001	18.69	1994-10-03	2008	1007
3006	1098.16	1994-10-04	2008	1007

#.13 WRITE A QUERY TO FIND ALL ORDERS ATTRIBUTED TO SALESPEOPLE IN LONDON

```sql
SELECT o.onum, o.amt,
o.odate, s.sname, s.city FROM Orders o
JOIN Salespeople s ON o.snum = s.snum
WHERE s.city = 'London';
```

Result Grid | Filter Rows: | Export:

onum	amt	odate	sname	city
3003	767.19	1994-10-03	Peel	London
3008	4723.00	1994-10-05	Peel	London
3011	9891.88	1994-10-06	Peel	London
3002	1900.10	1994-10-03	Motika	London

#12.WRITE A QUERY TO FIND OUT ALL ORDERS THAT ARE GREATER THAN THE AVERAGE FOR OCT 4TH

```sql
SELECT *FROM Orders
WHERE amt > (SELECT AVG(amt)
    FROM Orders
WHERE odate = '1994-10-04'
);
```

Result Grid | Filter Rows:

onum	amt	odate	cnum	snum
3002	1900.10	1994-10-03	2007	1004
3005	5160.45	1994-10-03	2003	1002
3008	4723.00	1994-10-05	2006	1001
3009	1713.23	1994-10-04	2002	1003
3011	9891.88	1994-10-06	2006	1001

#14.WRITE A QUERY TO FIND ALL THE CUSTOMERS WHOSE CNUM IS 1000 ABOVE THE SNUM OF SERRES

```sql
SELECT *
FROM Cust
WHERE cnum = (
    SELECT snum + 1000
    FROM Salespeople
    WHERE sname = 'Serres');
```

Result Grid | Filter Rows:

cnum	cname	city	rating	snum
2002	Giovanne	Rome	200	1003

#15.WRITE A QUERY TO COUNT CUSTOMERS WITH RATINGS ABOVE SAN JOSE’S AVERAGE RATING

```sql
SELECT COUNT(*) AS Customer_Count
FROM Cust
WHERE rating > (SELECT AVG(rating)
FROM Cust WHERE city = 'San Jose');
```

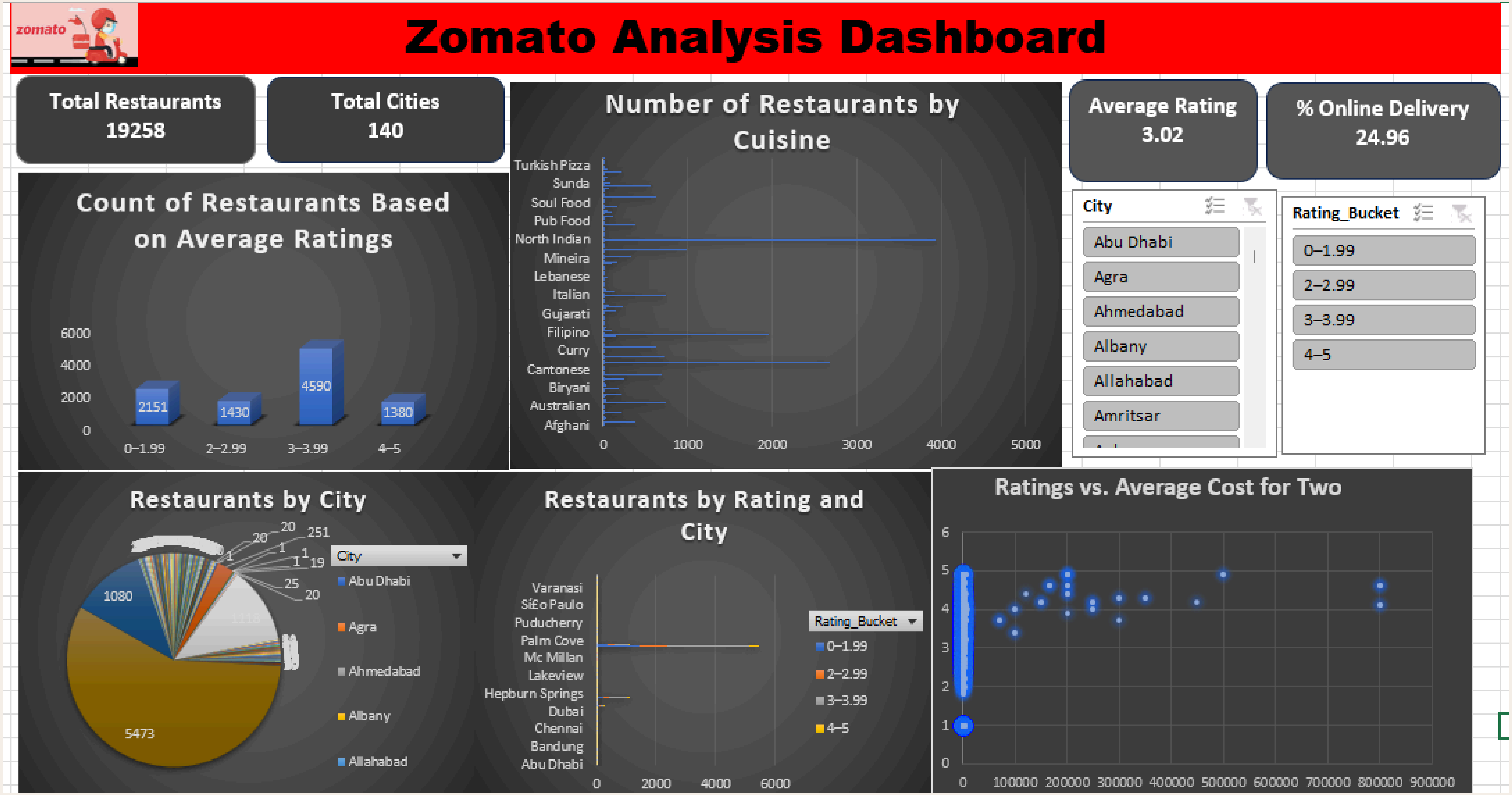
Customer_Count
0

#16.WRITE A QUERY TO SHOW EACH SALESPERSON WITH MULTIPLE CUSTOMERS

```sql
SELECT s.sname AS Salesperson,
COUNT(c.cnum) AS Customer_Count FROM Salespeople s
JOIN Cust c ON s.snum = c.snum
GROUP BY s.sname
HAVING COUNT(c.cnum) > 1;
```

Salesperson	Customer_Count
Serres	2
Rafkin	2

Excel Dashboard



Powerbi Dashboard

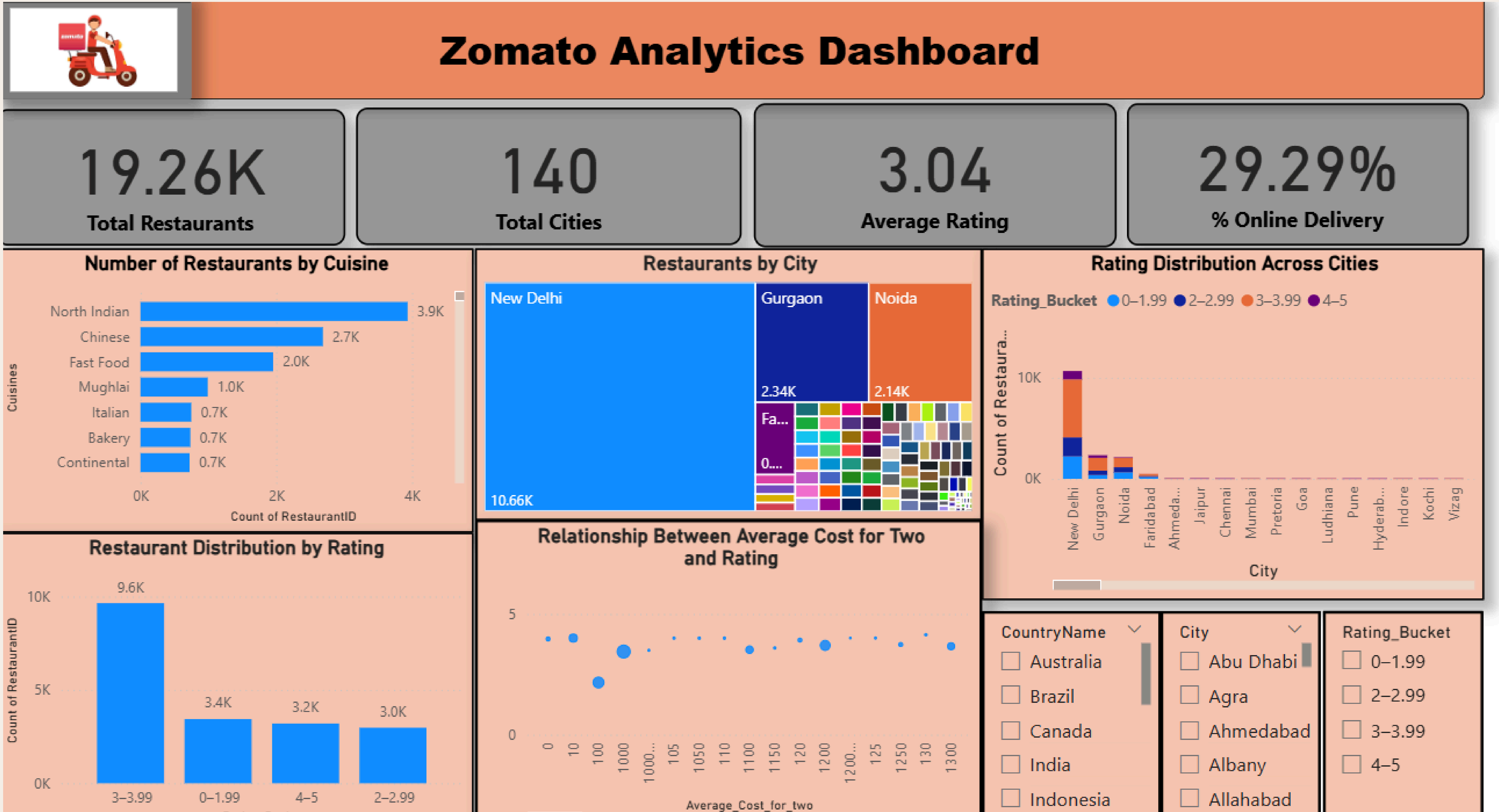
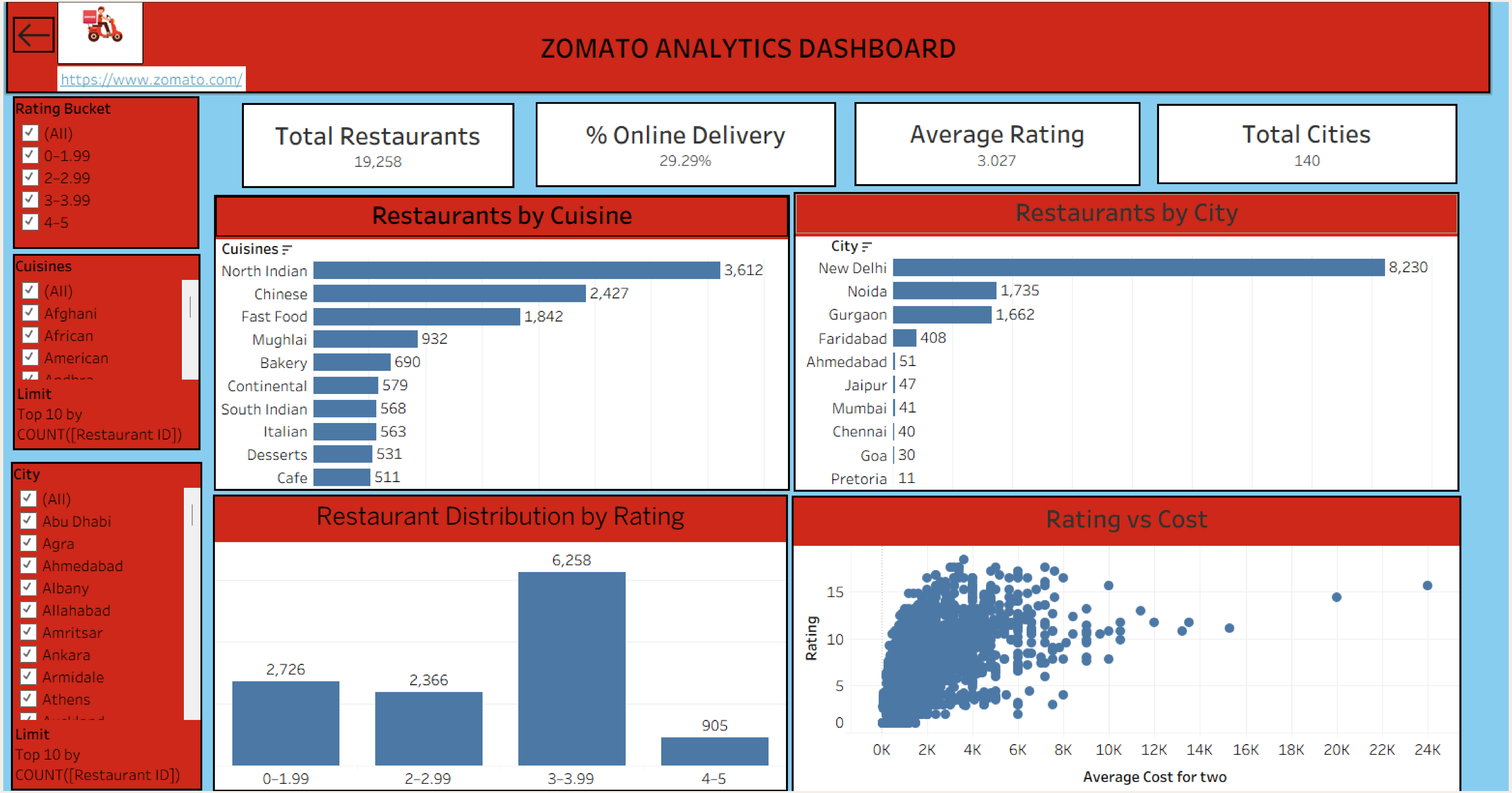


Tableau Dashboard



Final Insights

- North Indian cuisine is most popular
- Metro cities have highest restaurant count
- Most restaurants have average ratings (3-3.99)
- Higher cost does not always mean higher rating
- About 29% restaurants offer online delivery
- MySQL case studies helped us analyze employee performance and sales data using SQL joins and aggregation, improving our backend data analysis skills.

Key Learnings from the Project

This project strengthened our ability to approach data problems from raw data to actionable business insights.

- Handling raw data efficiently
- Selecting meaningful KPIs
- Choosing the right visualization
- Converting data into business insights
- End-to-end analytical thinking

Conclusion

In this project, we analyzed the Zomato dataset using Excel, Power BI, and Tableau. We cleaned the data, created KPIs, and built interactive dashboards to understand restaurant trends.

We found that most restaurants fall in the medium rating range, popular cuisines like North Indian and Chinese dominate, and higher cost does not always mean higher rating.

We started with MySQL case studies to understand data structure and perform basic analysis.

THANK YOU.